*Lúdica*

# Manual do Desenvolvedor

*Atualização da Sprint 3*

Este manual orienta a instalação, a manutenção e a implantação do Lúdica. A aplicação reúne uma interface React, uma API Node.js e um banco PostgreSQL. A versão da Sprint 3 permite navegar pelo catálogo, consultar desempenho e amizades e manter perfil e avatar. A execução dos jogos e a gestão pedagógica completa permanecem em evolução.

## Conteúdo do manual

1 Arquitetura e implantação

2 Instalação com Docker

3 Banco de dados e seed

4 Desenvolvimento e verificação

5 Modelo lógico e dados

6 API e regras de negócio

7 Deploy e operação

8 Evolução e referências

# 1 Arquitetura e implantação

O frontend é compilado pelo Vite em uma etapa com Node 22 Alpine e servido pelo Nginx. A API Express é executada em outro container Node 22 Alpine. O PostgreSQL mantém os dados em volume nomeado. O navegador executa o React e chama a API diretamente; o Nginx atual não faz proxy da API.

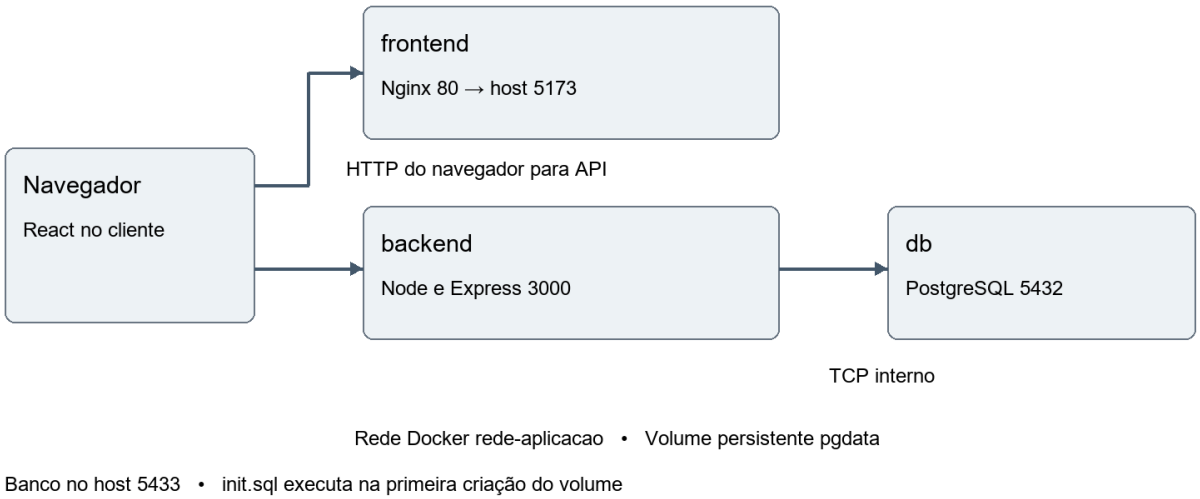



Figura 1 - Implantação atual definida no Docker Compose

| Serviço | Endereço local | Comunicação interna |
|---|---|---|
| frontend | http://localhost:5173 | Porta 80 no container |
| backend | http://localhost:3000 | Porta 3000 |
| db | localhost:5433 via cliente SQL | db:5432 para a API |

O banco não é um serviço HTTP. O backend aguarda o healthcheck do banco; o frontend depende do início do backend, sem verificação de prontidão da API. A rede bridge recebe o nome lógico rede-aplicacao.

# 2 Instalação com Docker

Pré-requisitos: Git e Docker com o comando Compose v2 disponível.

Docker deve estar iniciado, com suporte a containers Linux e acesso ao registro das imagens. Reserve as portas 5173, 3000 e 5433. Node não é necessário no host para a execução totalmente conteinerizada.

1. Verifique as ferramentas e clone o repositório.

2. Use a branch com as alterações da Sprint 3. Confira o commit e as alterações locais antes de trabalhar.

3. Inicie a aplicação a partir da raiz, apontando explicitamente para o Compose na pasta Docker.

```
git --version
docker --version
docker compose version
git clone https://github.com/marceloareas/Ludica.git
cd Ludica
git switch feat/parte-sprint3
git status
git rev-parse --short HEAD
docker compose -f Docker/docker-compose.yml config
docker compose -f Docker/docker-compose.yml up -d --build
docker compose -f Docker/docker-compose.yml ps
```

Alternativa equivalente: entrar em Docker e executar docker compose up -d --build. Executar esse comando na raiz sem -f não encontra o arquivo da cópia atual. Os Dockerfiles instalam as dependências com npm ci; não é preciso instalar pacotes no host.

### Configuração efetivamente usada

O Compose fornece DB_HOST=db, DB_PORT=5432, DB_USER=usuario_admin, DB_NAME=Ludica_desenv e a senha de desenvolvimento ao backend. O db.js também aceita essas variáveis. DATABASE_URL não é usada pelo código atual; arquivos .env isolados não são carregados automaticamente pela API.

As chamadas do frontend usam http://localhost:3000 diretamente. Criar apenas VITE_API_URL em .env não muda essas chamadas. JWT_SECRET é lida pela API, mas não está definida no Compose atual; configure uma chave própria antes de qualquer publicação.

# 3 Banco de dados e seed

Docker/init.sql cria as seis tabelas quando o PostgreSQL inicializa um volume vazio. Reiniciar os containers não reaplica o script em um banco existente. Alterações de estrutura exigem migração SQL revisada e backup prévio.

### Preparação de demonstração

O seed apaga os dados das seis tabelas com TRUNCATE e reinicia os identificadores. Execute somente em banco descartável de desenvolvimento ou demonstração. Ele não é uma migração e não deve rodar no banco de produção.

```
docker compose -f Docker/docker-compose.yml exec backend npm run seed
```

| Conta | Login por e-mail | Perfil |
|---|---|---|
| alice | alice@email.com | Aluno |
| bob e carol | bob@email.com e carol@email.com | Aluno |
| david | david@email.com | Aluno |
| prof_ana | ana@email.com | Professor |
| admin | admin@email.com | Administrador |

Todas as contas de demonstração usam a senha 123456. O seed cria Math Challenge, Memory Master, Typing Speed e Xadrez, pontuações, históricos e amizades. As URLs game1.com a game4.com são dados de exemplo. O seed inclui avatar para Alice, Bob, Carol, David e Admin; não inclui avatar para prof_ana.

### Verificação do esquema

```
docker compose -f Docker/docker-compose.yml exec db psql -U usuario_admin -d Ludica_desenv -c "\dt"
```

```
docker compose -f Docker/docker-compose.yml exec db psql -U usuario_admin -d Ludica_desenv -c "SELECT nome_usuario, tipo_usuario, perfil_usuario FROM usuario;"
```

A API verifica Gamertag com LOWER e trim. Embora anteriormente o relatório da Sprint 3 cite índice único case-insensitive, o init.sql da cópia consultada contém apenas UNIQUE simples em nome_usuario. Antes de garantir unicidade em requisições concorrentes, revise duplicidades e adicione uma migração com índice único em LOWER(nome_usuario). Não atribua essa garantia ao esquema atual.

# 4 Desenvolvimento e verificação

Escolha uma modalidade: totalmente conteinerizada, descrita na seção 2, OU híbrida, descrita abaixo. São alternativas mutuamente exclusivas, não etapas consecutivas. Não inicie backend e frontend no host enquanto os respectivos containers ocuparem as portas 3000 e 5173.

Para editar com recarga automática, mantenha apenas o banco em Docker e execute API e Vite no host. Use Node 22 compatível com os Dockerfiles e os lockfiles. Não execute essa modalidade junto dos containers backend e frontend nas mesmas portas.

```
docker compose -f Docker/docker-compose.yml stop backend frontend
```

```
docker compose -f Docker/docker-compose.yml up -d db
```

Em um terminal, a partir da raiz:

```
cd back-end
```

```
npm ci
```

```
npm run dev
```

Em outro terminal, a partir da raiz:

```
cd front-end
```

```
npm ci
```

```
npm run dev
```

A API usa localhost:5433 por padrão no host. Caso a configuração tenha mudado, defina as variáveis DB_* no processo. Para popular nessa modalidade, execute npm run seed em back-end, respeitando o aviso de apagamento de dados.

### Validação da entrega

```
cd front-end
```

```
npm run build
```

```
npm run lint
```

# 4 Orientação para o Teste de demonstração

### Preparação para apresentação e gravação

Antes do Playwright, escolha a modalidade de execução e confirme que frontend, API e banco estão disponíveis. O slide inicial do teste não inicia os serviços. Abra http://localhost:5173 e confira o login. Na modalidade com Docker, use os comandos abaixo na raiz; execute o seed somente se for necessário restaurar o banco descartável da demonstração.

```
docker compose -f Docker/docker-compose.yml ps
docker compose -f Docker/docker-compose.yml exec backend npm run seed
```

O seed apaga e reinicializa os dados. Não execute em banco com informações a preservar. Na modalidade híbrida, mantenha o banco em Docker e os dois terminais locais ativos; em um terceiro terminal, entre em back-end e execute npm run seed. Não use exec backend quando esse container estiver parado.

Confirme as contas Alice (alice@email.com), Professor (prof_ana ou ana@email.com) e Admin (admin ou admin@email.com), com senha de demonstração 123456. Confira o avatar e os dados do catálogo antes da gravação. Instale as dependências de teste e o Chromium na raiz:

```
npm ci
npx playwright install chromium
npx playwright test tests/e2e/demo-apresentacao.spec.js --headed --workers=1
```

### Timeout de navegação

Se a navegação exceder o tempo limite, verifique primeiro se http://localhost:5173 abre e se a API responde. Se os serviços não estiverem disponíveis, corrija o ambiente antes de alterar timeouts. Se a página estiver acessível e a espera por load for a causa, avalie o ajuste no arquivo tests/e2e/demo-apresentacao.spec.js:

```
await page.goto(URL, {
  waitUntil: 'domcontentloaded',
  timeout: 30000,
});
await expect(heading('Entrar')).toBeVisible();
```

Adapte URL e heading aos nomes usados no teste. Preserve uma asserção de prontidão da tela depois da navegação. O timeout de navegação não substitui os limites das asserções e do teste; revise-os se houver pausas deliberadas. Esse trecho é uma orientação de correção e não registra uma nova execução bem-sucedida.

O teste de demonstração tem pausas e pode preparar o avatar. Confira o arquivo antes de executá-lo. O script npm test do backend é apenas um marcador que termina com erro; não representa uma suíte unitária implementada.

Critérios de aceite utilizados:

- login de Alice por e-mail;
- Home, busca e catálogo;
- detalhe de Math Challenge com histórico e ranking;
- salvar perfil e confirmar após novo login;
- rejeitar BOB como Gamertag duplicada;
- salvar e recuperar avatar;
- comparar as telas de prof_ana e admin.

# 5 Modelo lógico e dados

O modelo anexado é preservado abaixo como referência de projeto. O SQL atual usa os nomes no singular e amplia os campos de usuário e jogo.

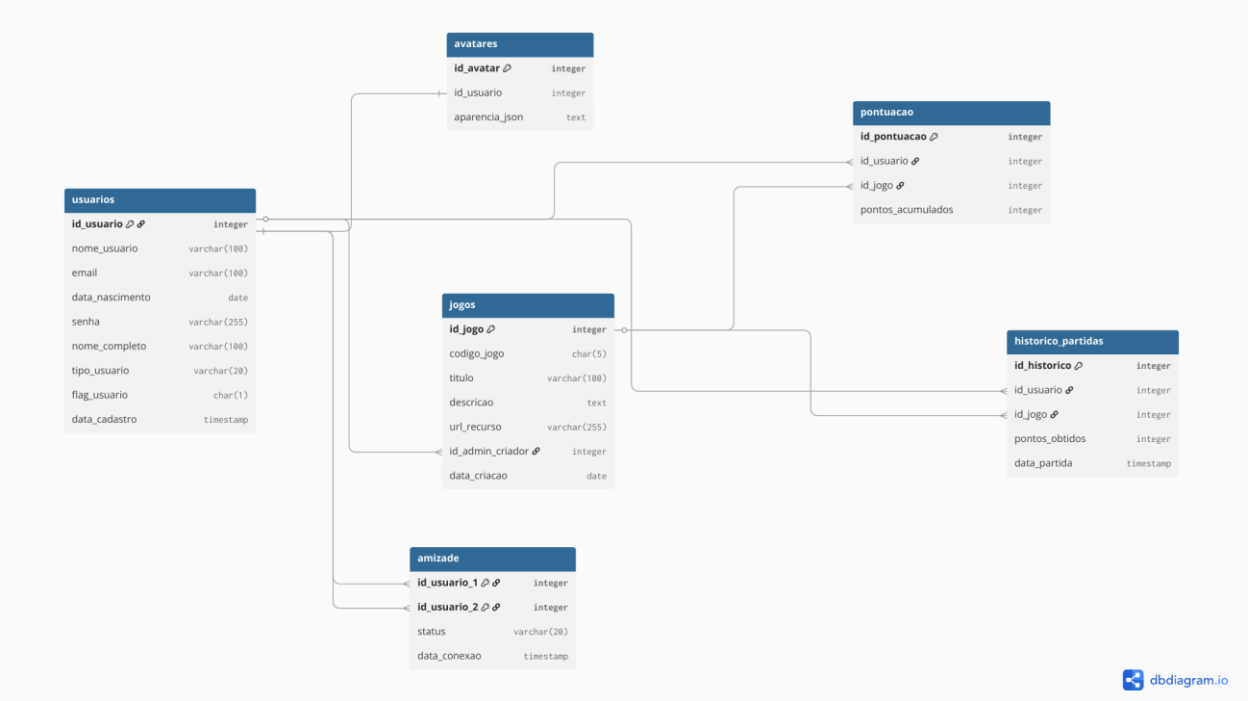



Figura 2 - Modelo lógico de referência anterior à Sprint 3

| Tabela atual | Chave e relacionamento | Campos e finalidade |
|---|---|---|
| usuario | PK id_usuario | Identidade, e-mail, senha hash, tipo_usuario, perfil_usuario, flag e datas |
| avatar | PK id_avatar; FK id_usuario UNIQUE | Configuração visual em aparencia_json TEXT |
| jogo | PK id_jogo; FK id_admin_criador | Código, título, descrição, URLs, idade, tempo e jogadores |
| pontuacao | PK id_pontuacao; FKs usuário e jogo | Pontos acumulados |
| historico_partida | PK id_historico; FKs usuário e jogo | Pontos obtidos e data por registro |
| amizade | PK composta dos dois usuários; duas FKs | Status e data da conexão |

### Campos adicionais no esquema efetivo

Em comparação ao Modelo Lógico original (Figura 2), nesta versão à usuario acrescenta perfil_usuario VARCHAR(20), com padrão Aluno e CHECK para Aluno ou Professor, aceitando NULL.

Para jogo acrescenta-se tempo_estimado INT, faixa_etaria VARCHAR(20), quantidade_jogadores INT e url_imagem TEXT. Esses campos pertencem ao SQL efetivo, mesmo sem aparecer no desenho anterior.

Relações:

- um usuário tem no máximo um avatar vinculado;
- usuário e jogo possuem vários registros de pontuação e histórico;
- um usuário pode criar vários jogos;
- amizade representa um autorrelacionamento entre dois usuários.
- As FKs de várias tabelas aceitam NULL no SQL atual. pontuacao não tem restrição UNIQUE sobre o par usuário e jogo.

# 6 API e regras de negócio

| Área | Rotas principais | Comportamento |
|---|---|---|
| Conta | POST /users/register e /users/login | Cadastro e JWT; login por e-mail ou Gamertag |
| Perfil | PUT /users/:id e /users/:id/password | JWT e autorização do proprietário |
| Avatar | GET e POST /avatars/:id | Escrita autenticada do próprio avatar |
| Catálogo | GET /games?id_usuario=ID e /games/:id | Catálogo personalizado e detalhe |
| Histórico | POST /history e GET /history/:id_usuario/:id_jogo | Registro e consulta por usuário e jogo |
| Social | /friends/enviar, /aceitar, /recusar e /delete | Convites e manutenção de vínculo |
| Ranking social | GET /friends/ranking/:id_usuario/:id_jogo | Aluno e amigos aceitos; melhor pontuação |

- tipo_usuario distingue Usuario e Administrador;
- perfil_usuario distingue Aluno e Professor.
- A constraint permite NULL, utilizado no seed do Admin.
- Professor não participa de partidas na regra de historyService.
- O ranking social filtra alunos e exclui Administrador e Professor; ordena pela melhor pontuação no histórico, não pelo saldo de pontuacao.
- RN01 é sustentada por UNIQUE em avatar.id_usuario.
- RN03 e RN04 são refletidas no ranking de amigos aceitos e no fluxo de convites.
- RN02 define gestão exclusiva do catálogo pelo Administrador, porém POST /games não possui middleware de autenticação ou autorização nas rotas atuais.
- RN05, alteração de assuntos e associação a vários assuntos, não tem tabelas e CRUD correspondentes no esquema atual.

### Limites relevantes para manutenção

Perfil, senha e gravação de avatar têm proteção explícita por JWT e proprietário. Rotas de leitura de avatar, histórico, pontuação, amizade e catálogo não demonstram a mesma proteção no arquivo de rotas. Não apresente a privacidade de toda a API como garantida. A UI usa identidade armazenada em localStorage, que não substitui autorização no servidor.

Mais populares exibe até seis jogos na ordem recebida por id_jogo; não calcula popularidade por partidas. Continue jogando usa a última data de histórico do usuário e mostra até quatro jogos. Os filtros curriculares não recebem seus campos no SELECT atual e podem ficar desabilitados.

# 7 Deploy e operação

### Implantação local reproduzível

A construção com Compose produz frontend estático servido por Nginx, API e banco. Esse é o procedimento de implantação local entregue pelo projeto. git push publica código no repositório e, sozinho, não faz deploy de containers.

```
docker compose -f Docker/docker-compose.yml up -d --build
docker compose -f Docker/docker-compose.yml ps
docker compose -f Docker/docker-compose.yml logs --tail=100 backend db frontend
```

Abra a página, faça login e recarregue /home diretamente para conferir o fallback do React Router. Teste o catálogo pela API com o ID obtido no login; GET / não é uma rota de healthcheck da API. Não use o seed como etapa de atualização de um ambiente com dados reais.

### Publicação em servidor com acesso remoto

1. Prepare um servidor com Docker Compose, domínio, certificado TLS, firewall e espaço persistente para o banco. Escolha uma revisão aprovada do código.

2. Substitua as chamadas localhost:3000 do frontend por uma URL de API acessível aos navegadores ou por /api. Na segunda opção, configure um proxy reverso para backend:3000 removendo o prefixo /api ao encaminhar. Recompile o frontend.

3. Configure credenciais próprias do banco em db e backend e injete JWT_SECRET por ambiente ou mecanismo de segredos. As credenciais do Compose são de desenvolvimento. Restrinja CORS às origens autorizadas.

4. Proteja no backend as rotas ainda abertas, especialmente criação de jogos, pontuações, histórico e amizades. Autorize por identidade e perfil do JWT. Resolva a unicidade da Gamertag no banco e valide migrações.

5. Use um arquivo Compose de produção revisado com as mesmas relações de build, rede e persistência; mantenha o banco sem porta pública e exponha o tráfego web pelo proxy TLS. Fixe versões das imagens, evitando postgres:latest em produção.

6. Execute docker compose -f Docker/docker-compose.producao.yml config e depois up -d --build somente após criar esse arquivo e revisar suas configurações. Esse arquivo não acompanha o repositório atual.

7. Valide login, gravação do perfil, avatar, catálogo e rotas profundas a partir de outra máquina. Confirme backup e restauração antes de liberar acesso.

A versão atual precisa dessas adequações para implantação remota. Não há hospedagem de produção, pipeline automático de deploy ou mecanismo completo de segurança comprovado no código consultado.

# 8 Operação e continuidade

### Backup e restauração

A partir da raiz, faça um backup antes de atualização ou migração. O exemplo usa arquivo dentro do container e cópia para o host para evitar problemas de redirecionamento binário no PowerShell.

```
docker compose -f Docker/docker-compose.yml exec db pg_dump -U usuario_admin -d Ludica_desenv -Fc -f /tmp/ludica.dump
```

```
docker compose -f Docker/docker-compose.yml cp db:/tmp/ludica.dump ./ludica.dump
```

Restaure em banco de teste isolado, com credenciais e schema de destino preparados: copie o arquivo para /tmp/ludica.dump no container e use pg_restore -U usuario_admin -d Ludica_desenv --clean --if-exists /tmp/ludica.dump. Esse comando substitui objetos e dados do destino; reserve-o para restauração deliberada. Faça também backup de configuração e da revisão do código.

Para parar sem apagar o volume: docker compose -f Docker/docker-compose.yml down. Não use down -v em banco que precisa ser preservado. init.sql só é reaplicado em volume novo; corrigir o arquivo não migra um volume já criado.

### Diagnóstico e atualização

Se a Home não abrir, confira Docker e porta 5173. Se o login falhar por conexão, confira API em 3000 e logs. Se houver erro SQL, confira schema, volume e variáveis DB_*. Se o avatar não aparecer, salve uma configuração válida e autentique novamente. Uma falha na leitura do avatar não deve impedir o login.

Fluxo Git: git status; criar branch de trabalho; revisar diff; adicionar somente arquivos da mudança; commit; push da branch; revisão e integração. Antes de atualizar containers, registre a revisão anterior e faça backup. Para voltar uma versão, reconstrua a revisão anterior; uma mudança de schema exige plano próprio de reversão.

### Planejamento da Sprint 4

O feedback de 07/10 solicita indicadores do jogo, como número de partidas, jogos mais jogados e tempo médio, com lógica de cálculo e controle do Administrador; CRUD do catálogo e disponibilização de acesso; CRUD de atividades dentro do jogo para Professor e Administrador; loja de itens de avatar e sistema de moedas. Esses recursos exigem definição de dados, autorização, regras de cálculo e testes. O pedido de alteração da fonte refere-se à apresentação.

Continuam no roadmap: conquistas, multiplayer em tempo real, desafios diários, WebSockets, ampliação de TypeScript e CI/CD. Já há testes E2E em Playwright, embora os documentos antigos tratassem testes automatizados integralmente como trabalho futuro.

Referências documentais: Manual do Desenvolvedor e Manual do Usuário anteriores, de 27/06/2026; Modelo Lógico.pdf; Containers.docx; Regras de Negócio.xlsx; Documento de Trabalho Futuro, de 27/06/2026.

Código de referência: github.com/marceloareas/Ludica, cópia Ludica_Remoto/Ludica, branch feat/parte-sprint3, HEAD 11704fe. Arquivos centrais: Docker/docker-compose.yml, Docker/init.sql, Dockerfiles, back-end/seed.js, back-end/src e front-end/src. As instruções descrevem o conteúdo da cópia consultada; a execução deve ser validada no ambiente de destino.

Feedback do Professor registrado em 07/10/2026: evolução de indicadores e CRUD do Administrador, disponibilização de acesso ao jogo, CRUD de atividades de Professor e Administrador e loja de avatar com moedas. Entrega da documentação em 09/10/2026 na página do Lúdica no Teams.